

Objetivo

Entraste como **Product Builder** a Bibliotk, una plataforma de reseñas de libros. Perfecta semana para llegar: hace tres días un autor pagó una campaña de reseñas falsas, el promedio de su libro pasó de 2.1 a 4.9 estrellas en cuatro horas, y mientras eso ocurría la home tardó 30 segundos en cargar. El sistema de moderación baneó las cuentas al día siguiente, pero el promedio del libro sigue en 4.9.

Tu trabajo es rehacer el motor de calificación: que los promedios sean correctos, que sigan siendo correctos cuando alguien es baneado después de haber reseñado, y que todo esto aguante cuando la próxima campaña llegue — porque va a llegar.

Y una cosa más: nadie en Bibliotk tiene claro qué debería pasar con ese libro que quedó en 4.9, ni qué se le dice al autor, ni qué ve el lector mientras tanto. Esa parte también es tu trabajo.

Brief

Los usuarios escriben reseñas de libros. Una reseña tiene una calificación en estrellas y, opcionalmente, un texto. Cada libro muestra el promedio de sus calificaciones.

Los usuarios pueden ser baneados por el equipo de moderación, y casi siempre eso ocurre *después* de que ya escribieron sus reseñas. Un usuario baneado puede tener miles de reseñas repartidas por el catálogo.

La home lista 50 libros con su promedio. Los títulos populares superan las 500.000 reseñas.

Lo que te llega en tu primera semana

Cinco mensajes que están en tu bandeja el día que entras:

- **Moderación:** "Antes de banear a alguien con 4.000 reseñas necesito saber cuánto le va a mover el promedio a cada libro. Hoy baneo a ciegas y después me llegan los reclamos."
- **Growth:** "El cartel de 'Reseñas Insuficientes' en la home nos está matando el click-through. ¿No podemos mostrar el promedio igual, aunque sea con dos reseñas?"
- **Un autor, vía soporte:** "Mi libro bajó de 4.6 a 2.3 de un día para otro y nadie me avisó nada. Exijo una explicación."
- **Soporte:** "Tengo 12 tickets de usuarios preguntando por qué su reseña 'ya no aparece'. No sé qué responderles."
- **Dirección:** "Que no se vuelva a repetir." Sin más detalle.

No todos tienen razón y no todos piden lo mismo. Parte de lo que evaluamos es qué haces con cada uno.

Tareas

- Implementa el sistema en Ruby on Rails. Solo backend: no se necesita frontend. Cómo expones el sistema hacia afuera es decisión tuya, mientras un cliente pueda consumirlo.
- **Reseñas:**
 - Una reseña tiene una calificación entera de 1 a 5 estrellas y un contenido de texto opcional de máximo 1000 caracteres.
 - Un usuario puede dar como máximo una reseña por libro.
 - Un usuario puede editar el rating y el contenido de su reseña, y puede eliminarla.
- **Cálculo del promedio:**
 - El promedio se expresa con un decimal, con redondeo half-up (3.25 → 3.3).
 - Un libro con menos de 3 reseñas muestra "Reseñas Insuficientes" en lugar del promedio.
 - Las reseñas de usuarios baneados no cuentan para el promedio.
 - El sistema debe ser capaz de mostrar los 50 libros con su puntaje promedio de manera eficiente, garantizando que el consumo de recursos de la base de datos no crezca de forma proporcional a la cantidad de interacciones de los usuarios.
- **Baneos:**
 - Se puede banear y desbanear usuarios.
 - Banear o desbanear debe quedar reflejado en los promedios de todos los libros que ese usuario reseñó.
- **Concurrencia y consistencia:**
 - Si 200 usuarios distintos reseñan el mismo libro simultáneamente, al terminar el promedio y el conteo de ese libro deben ser correctos. No basta con que "casi siempre" lo sean.
 - Registrar una reseña debe responder rápido.
- **Producto** (esto no debería agregar más de una hora de código):
 - Decide qué pasa con la reseña de un usuario baneado desde el punto de vista de cada actor: ¿desaparece del libro, queda visible pero no cuenta, la sigue viendo quien la escribió? Tu API debe reflejar esa decisión de forma explícita, no como efecto colateral de la implementación.
 - Resuelve la pregunta de Moderación: expón alguna forma de saber qué impacto tendría banear a un usuario **antes** de banearlo, sin escribir en producción. Si tu arquitectura está bien hecha esto es barato; si te sale caro, eso también es un hallazgo y lo documentas.
 - Define las 3 métricas o eventos mínimos que instrumentarías para saber (a) que el motor de promedios está sano y (b) que una campaña está ocurriendo ahora, no tres días después. No hace falta implementarlas, pero sí decir dónde irían y quién las mira.
 - Decide qué hacer con el libro que quedó en 4.9 y con los que hoy están mal y nadie notó: ¿se corrige solo, necesita un backfill, quién lo dispara, qué pasa mientras corre?
 - Contéstale a Growth y al autor. Una decisión de producto puede ser "no", siempre que puedas sostenerla.
- **Tests:**
 - Usa RSpec.

- Todo comportamiento no trivial debe tener un test que falle sin tu implementación.
- Como mínimo, cubre: el redondeo en sus bordes, el umbral de las 3 reseñas, el baneo retroactivo, el ciclo editar/eliminar, y el invariante de unicidad bajo concurrencia.

Entregables

1. El código, en un repositorio con instrucciones para levantarlo y correr los tests.
2. Un **DECISIONES.md** breve con:
 - Los requisitos de este enunciado que te resultaron ambiguos o contradictorios, y qué decidiste en cada caso.
 - Los trade-offs que tomaste y qué costo tiene cada uno.
 - Qué dejarías fuera si esto saliera a producción mañana, y qué harías distinto con una semana más.
3. Un **PRODUCTO.md** breve (máximo 2 páginas) con:
 - Qué decidiste frente a cada uno de los cinco mensajes de la primera semana, y a cuál le dijiste que no.
 - Las métricas o eventos que definiste, y qué acción concreta desencadena cada señal.
 - El plan para los promedios que hoy están mal en producción: qué corres, en qué orden, y qué se comunica hacia afuera.
 - Una cosa del enunciado que cambiarías si este fuera tu producto — el umbral de 3 reseñas, el redondeo, la regla de baneos, lo que sea — y por qué.
4. Preferimos dos párrafos claros a diez bullets. Escríbelo pensando en que lo va a leer alguien que no vio tu código.

Bonus

- Un seed que genere un libro con 500.000 reseñas, y una medición que demuestre que el listado de la home no se degrada con él en el catálogo.
- Qué pasa si el mismo autor vuelve a comprar reseñas: ¿cómo detectarías la anomalía?

Uso de IA

Puedes usar asistentes de IA sin restricción. En la entrevista de revisión te vamos a pedir que:

- Expliques cualquier línea del código que entregaste y qué pasa si se elimina.
- Cuentas qué sugerencias de la IA descartaste y por qué.
- Defiendas una de tus decisiones de producto frente a alguien que opina lo contrario. Vamos a hacer el papel de Growth.

No evaluamos si usaste IA, sino si entiendes lo que entregaste.

Alcance

Si algo no alcanza a entrar, prefíerelo bien resuelto y documentado antes que completo y a medias: déjalo escrito en [DECISIONES.md](#).

De esas horas, alrededor de una es de producto: decidir y escribir, no código extra. Un motor sólido con un [PRODUCTO.md](#) honesto vale más que features de más.

Criterios de evaluación

- **Compleitud:** ¿implementaste lo pedido?
- **Correctitud:** ¿el sistema se comporta bien en los casos bordes, no solo en el camino feliz?
- **Criterio:** ¿tomaste decisiones defendibles?
- **Mantenibilidad:** ¿el código es claro y modificable por otra persona?
- **Testing:** ¿los tests prueban comportamiento real o solo acompañan al código?
- **Criterio de producto:** ¿distingue lo que le piden de lo que el usuario necesita? ¿sabe decir que no y sostenerlo?
- **Comunicación:** ¿se entiende lo que escribió sin que esté presente para explicarlo?

Éxito y buen código,

El equipo de ComunidadFeliz